

METHODOLOGIE DE DEVELOPPEMENT DE BASES DE CONNAISSANCES POUR LA SDF AVEC L'ENVIRONNEMENT OPEN-SOURCE "VISUAL FIGARO"

BUILDING KNOWLEDGE BASES IN THE DEPENDABILITY FIELD WITH THE OPEN-SOURCE ENVIRONMENT "VISUAL FIGARO"

Guillaume TORRENTE, Marc BOUISSOU
EDF R&D
1 avenue du général de Gaulle 92141 Clamart cedex France

Résumé

L'objectif de la communication est de présenter une démarche permettant de passer progressivement d'un squelette de base de connaissances composé du minimum permettant de construire des modèles de sûreté de fonctionnement avec l'outil de modélisation KB3 à un outil complet et convivial, muni de divers modes de visualisation, de contrôles de la saisie, et de traitements automatisés des modèles. Cette approche s'appuie sur l'environnement de développement Visual Figaro, qui peut aussi être utilisé pour la maintenance de bases de connaissances développées avec des outils antérieurs ; ce point est très important étant donné le nombre important et la longue durée de vie des bases de connaissances existantes.

Summary

The aim of this paper is to present a progressive approach enabling to pass from a skeleton of knowledge base, containing the minimum information necessary to build dependability models with the modelling tool KB3, to a complete and user-friendly tool, with input checking, various visualization modes, and automated processing. This approach relies on the Visual Figaro development environment, that can also be used for the maintenance of existing knowledge bases. This is an important point, given the large number and long life of existing knowledge bases.

Introduction

A EDF, les études de sûreté de fonctionnement ont dû faire face à une complexité importante des systèmes à étudier (systèmes nucléaires ou systèmes du réseau de transport). De multiples études réalisées depuis les années 1980 ont abouti à une expérience capitalisée dans la Plate-forme outils KB3, en constant développement depuis cette époque, et des bases de connaissances écrites dans le langage de modélisation FIGARO. Ce langage a été le premier créé spécialement pour faciliter la construction de modèles de sûreté de fonctionnement de systèmes [1]. Il a été développé en 1989-1990 à EDF R&D et validé depuis par des centaines d'études de systèmes complexes. C'est ce langage qui a permis à EDF de capitaliser dans des bases de connaissances à la fois pérennes et évolutives, la majorité des connaissances sur la sûreté de fonctionnement des systèmes hydrauliques, électriques, de contrôle-commande, de télécommunication étudiés au fil des ans.

Le langage FIGARO est tout ce qui est requis pour construire des modèles de sûreté de fonctionnement uniquement à l'aide de texte ; mais afin d'offrir à ses utilisateurs une interface homme-machine agréable et reposant fortement sur le graphique, l'outil KB3 a des exigences supplémentaires. Il faut paramétrer son interface graphique générique en définissant à minima les icônes et les styles de liens à associer aux types FIGARO. Mais on peut aller beaucoup plus loin en définissant aussi des variantes graphiques de ces éléments (avec des changements de couleurs, des affichages de texte variable...) qui seront appelées dans différents modes de visualisation qui pourront présenter des aspects variés d'un système, en facilitant ainsi beaucoup l'appréhension. Il est également possible de définir des contraintes sur les assemblages de composants permis, de façon à éviter que l'utilisateur peu au fait des concepts d'une base de connaissances donnée saisisse des modèles dénués de signification.

Dans cet article, nous allons montrer en nous appuyant sur un exemple de modélisation de système simple (que nous avons déjà traité dans l'article où nous avons comparé les langages de modélisation AltaRica et Figaro [2]) les différentes étapes du travail de construction d'un outil de modélisation d'une catégorie de systèmes (ici des systèmes électriques).

Présentation de la plate-forme outils KB3

KB3 est un logiciel permettant d'automatiser les études de sûreté de fonctionnement à partir d'une base de connaissances adaptée aux problématiques du système à étudier. A partir d'une représentation graphique de ce système, KB3 génère un modèle textuel dit en FIGARO 0, transparent pour l'utilisateur, qui peut soit être utilisé pour générer automatiquement un arbre de défaillances soit constituer le fichier d'entrée aux outils de traitement adaptés aux systèmes dynamiques (FIGSEQ et YAMS, décrits plus loin).

Autrefois, les modèles utilisés à EDF pour les études de fiabilité et disponibilité étaient construits à la main, ce qui présentait de multiples inconvénients, parmi lesquels une cohérence souvent imparfaite entre les hypothèses faites par des analystes différents et une faible traçabilité des hypothèses (source potentielle d'erreurs lors des réactualisations des études).

D'où les principaux avantages de KB3 qui sont :

- le gain de temps dans la réalisation des études,
- l'accessibilité de l'outil à des non spécialistes du fait de bases de connaissances dédiées à une application,
- la qualité des études obtenue par une garantie de cohérence entre les différents modèles et la traçabilité des données,
- la possibilité de faire plusieurs traitements à partir d'un modèle unique.

Principes de la plate-forme outils KB3

Pour réaliser l'étude de sûreté de fonctionnement d'un système avec KB3, il faut disposer d'une [base de connaissances](#) préalablement écrite en langage FIGARO. L'utilisateur de KB3 utilise donc à la fois une base de connaissances adaptée à son application et le logiciel KB3

lui-même qui est constitué d'une interface conviviale pour la description graphique du système étudié et des buts de l'étude. Cette interface permet la construction de modèles de systèmes industriels complexes.

A partir de cette description graphique dans l'IHM de KB3 et des connaissances décrites dans la base, le logiciel KB3 génère un modèle dit **FIGARO** à l'ordre 0. Ce modèle unique est utilisé :

- soit par le générateur d'arbres de défaillances pour les modèles statiques,
- soit par le générateur de séquences FIGSEQ pour les modèles dynamiques markoviens,
- soit par l'outil de simulation de Monte Carlo YAMS pour tous les modèles (qu'ils soient markoviens ou non).

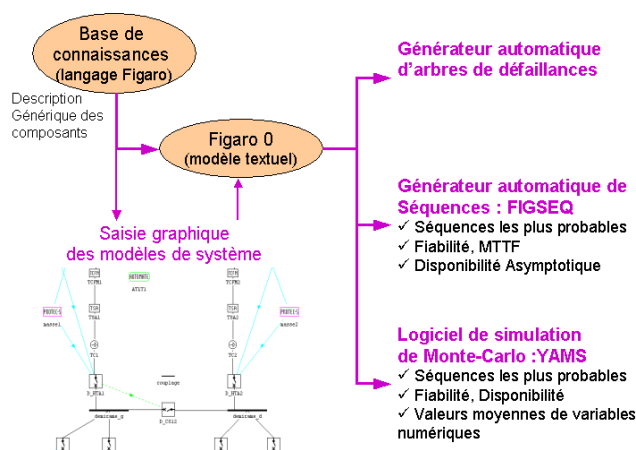
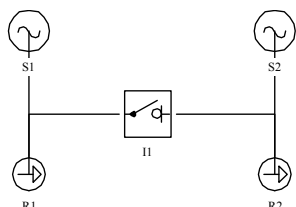


Figure 1 : Principes de la plate-forme outils KB3

La plate-forme outils KB3 a déjà fait l'objet de nombreuses publications, dont on trouvera une sélection dans la liste des références en fin d'article. La présentation ci-dessus est volontairement très succincte, de façon à laisser le plus de place à la description de la démarche de construction d'une base de connaissances s'appuyant sur Visual Figaro. Cette démarche va être illustrée par un problème concret : la construction d'une base de connaissances pour décrire des systèmes électriques tels que celui décrit dans la section suivante.

Présentation du cas-test

Soit le système non réparable suivant, composé de deux sources électriques (S1 et S2), deux récepteurs (R1 et R2) et un interrupteur (I1). Le récepteur R1 est en priorité alimenté par la source S1, et est réalimenté par la source S2 en cas de perte de S1, grâce à la fermeture de I1. Un fonctionnement symétrique est supposé pour l'alimentation de R2.



Mode de défaillance	en fonctionnement	à la sollicitation
Sources	Destruction : DS	
Récepteurs	Court-circuit : CC	
Interrupteur	Fermeture intempestive : FI	Refus de fermeture : RF

Figure 2 : système électrique simple – schéma et modes de défaillance

On suppose pour simplifier que les seuls modes de défaillance à prendre en compte sont ceux du tableau ci-dessus. Le mode de défaillance DS des sources peut se produire "spontanément", ou bien de manière provoquée par d'autres dysfonctionnements. La destruction provoquée des deux sources se produit si elles sont mises en relation par la fermeture de l'interrupteur alors qu'aucune d'elles n'est perdue. Si un court-circuit se produit sur un récepteur, cela provoque également la destruction des sources auxquelles le récepteur est relié. Ces hypothèses ne sont guère réalistes : dans un système réel, on aurait à la fois des modes de défaillances supplémentaires, à commencer par la coupure et le court-circuit sur les câbles, et des composants supplémentaires, tels que des disjoncteurs destinés à éviter la propagation de courts-circuits. En fait, ce cas-test est repris de l'article [2], dans lequel nous avons dû simplifier l'exemple à l'extrême pour pouvoir lister *intégralement* les modèles dans les deux langages FIGARO et AltaRica. Dans la suite du présent article, nous ne ferons qu'ébaucher la partie en langage FIGARO, pour laisser de la place aux opérations de paramétrage de l'interface graphique de KB3.

Démarche de construction d'une base de connaissances pour KB3

Généralités

La méthode de travail s'appuie fortement sur un environnement en open source appelé Visual Figaro, qui est disponible sur le site de téléchargement de KB3 [5]. L'objectif de cet environnement est de décharger au maximum l'utilisateur du travail de gestion de la cohérence entre différentes parties de la base de connaissances. Tant qu'il travaille uniquement en langage FIGARO, l'utilisateur peut d'un simple clic appeler un outil de contrôle syntaxique qui lui signale toute incohérence "statique" (c'est à dire détectable sans exécution d'un modèle). Puis lorsqu'il commence à paramétrer l'interface graphique, une IHM à menus et boutons lui propose chaque fois que cela est possible des listes de choix, en fonction du contenu des types définis en FIGARO.

Visual Figaro a été conçu de manière à être utile à tout concepteur de base de connaissances, *quel que soit son degré d'expertise* dans ce type de travail. Les aides syntaxiques sont discrètes et non invasives. Elles se présentent sous la forme d'expressions typiques que l'on peut choisir en fonction du contexte dans des menus arborescents et insérer dans le texte FIGARO.

Objectifs : tout ce qu'il faut faire

En s'appuyant sur cet outil, la méthode de développement d'une nouvelle base de connaissances passe par les étapes suivantes :

- définition de la structuration statique de l'ensemble des types FIGARO (leurs noms, leurs relations d'héritage, leurs interfaces),
- définition d'un paramétrage minimal de l'IHM de KB3 (choix d'icônes, de caractéristiques de liens, instructions de remplissage des interfaces)
- itérations sur :
 1. écriture des règles décrivant le comportement des types FIGARO
 2. test sur des petits exemples d'assemblages de composants
- ajout de variantes graphiques (changements de couleurs, affichage de textes prévus pour la simulation interactive)
- ajout de contraintes sur la saisie pour rendre l'outil créé robuste

L'ordre des étapes de cette méthode a été choisi de façon à minimiser les risques de retour en arrière, avec nécessité de modifier ce que l'on a déjà créé. Nous allons les reprendre en détail, avec des illustrations par des copies d'écran de VisualFigaro.

Détermination des types et de leurs interfaces

La première partie, préliminaire à l'étude de tout système, est la définition de tous les types et des interfaces entre ces types ; cela revient à peu près à créer un modèle entité-relation pour l'ensemble des objets du système. Dans le cas du système électrique étudié les types sont très simplement définis comme représentation directe des composants présents dans le circuit, auxquels on ajoute le type "nœud" afin d'y déclarer des caractéristiques communes aux sources, récepteurs et interrupteurs afin de maintenir la base aussi concise que possible en évitant de nombreuses redondances. Ces types récupéreront à leur tour ces propriétés en utilisant la relation d'héritage du langage FIGARO : ils hériteront donc tous du type « nœud ». C'est ce qui sera supposé dans le reste de l'article mais ce n'est qu'une des nombreuses possibilités de représentation du système.

Création des types dans Visual Figaro

Une fois cette première ébauche papier de la base de connaissances terminée l'écriture en langage FIGARO peut commencer. Le logiciel Visual Figaro va alors commencer à donner son plein potentiel. Son but principal est de fournir des "patterns" pour créer une base de connaissances. Il s'agit d'expressions typiques du langage FIGARO qui permettent à l'utilisateur de prendre connaissance de la syntaxe de FIGARO de manière progressive. Il a à tout moment la liberté d'explorer la syntaxe dans le but de mieux la connaître sans pour autant lire la documentation en entier. Les prototypes proposés peuvent en outre donner un axe de lecture pour la documentation complète de FIGARO dont la lecture reste cependant indispensable à long terme.

Suivant l'architecture de la base définie dans la phase initiale il est temps d'écrire les premiers éléments en langage FIGARO. Pour cela il est évidemment conseillé d'utiliser Visual Figaro qu'il faudra lancer à partir d'un logiciel tiers : l'éditeur de texte (en open source) jEdit. La figure 3 représente la fenêtre principale avec la racine d'une arborescence disposée sur la gauche de l'écran :

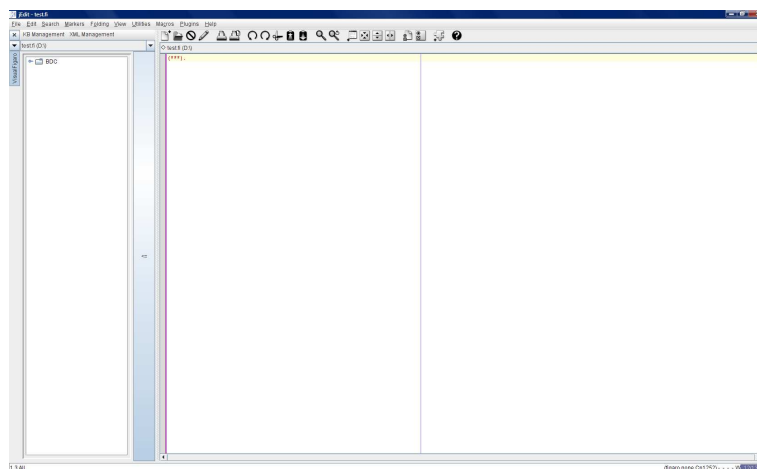


Figure 3 : Fenêtre principale de Visual Figaro

Avant de commencer la rédaction il est deux notions qu'il faut connaître à propos des bases de connaissances : la hiérarchie des types et la hiérarchie syntaxique. Une base de connaissances contient en effet deux types de hiérarchie : celle des types (c'est elle qui définit les liens d'héritage entre composants du système), et la hiérarchie qui reflète simplement l'arbre syntaxique de la syntaxe FIGARO d'une base de connaissances. Dans notre exemple, les types source, interrupteur et récepteur vont hériter du type nœud : ceci est représentatif de la hiérarchie des types et sera visible sous une forme graphique dans une fenêtre spécifique que l'utilisateur pourra afficher à tout moment (cette fonction n'est pas encore disponible, mais elle est prévue). Cependant la fenêtre de travail utilisée le plus souvent est celle qui correspond à l'arbre syntaxique.

Par exemple, lors de l'écriture d'une base de connaissances toute déclaration de type doit suivre une certaine hiérarchie pour encapsuler les règles, les constantes et les autres structures propres au type déclaré. Au plus haut niveau se trouvent les types puis immédiatement en-dessous se trouvent les constantes, les règles, puis au sein des règles, par exemple, se trouvent les prémisses et les conclusions de ces règles, au sein desquelles se trouvent les variables qui entrent en jeu, etc. Toutes ces informations sont en outre réunies sous une racine nommée BDC afin de maintenir l'unité de la base. Dans la base étudiée la prémisse « fonction = source » appartient à la règle « regle_tension_source » qui appartient à son tour au type « nœud » qui appartient à la base de connaissances. C'est cette hiérarchie qui est la plus utile lors de l'écriture de la base et c'est elle qui est représentée dans la partie gauche de la fenêtre sous la forme d'un arbre. Le principe fondamental est que **chaque étage de l'arbre syntaxique de Visual Figaro propose une série de prototypes de codes pour les parties plus profondes dans l'arbre. Ces prototypes sont accessibles à l'aide d'un clic droit sur la partie correspondante de l'arbre.** Le fait d'en sélectionner un le copie dans le presse-papier, ce qui permet de l'utiliser dans la fenêtre de droite en prenant soin de remplacer les noms génériques par les noms appropriés.

L'écriture de la base commence donc par la création du code des entités qui se trouvent au plus haut niveau dans la hiérarchie syntaxique c'est-à-dire les types. Leur création suit une syntaxe rigoureuse en langage FIGARO. Même si celle-ci a été pensée pour être la plus simple à utiliser, la complexité des notions exprimées la rend tout de même difficile au premier abord. C'est maintenant que Visual Figaro va pouvoir aider le développeur de base de connaissances. Des bouts de code représentant des déclarations de types vont être proposés à travers l'interface graphique et simplifier considérablement la recherche et l'apprentissage de la syntaxe. Les types se situant au plus haut niveau, ils se trouvent juste sous la racine BDC et les informations à leur propos s'obtiennent donc en effectuant un clic droit sur la feuille BDC de l'arbre.

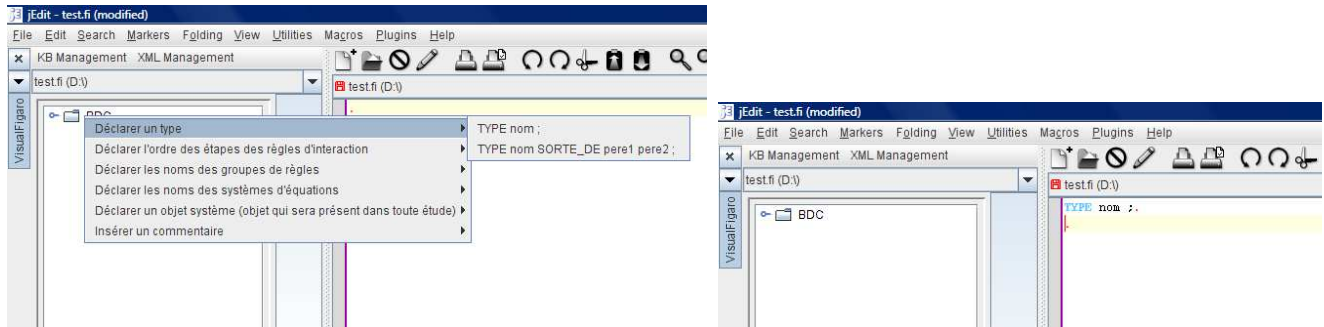


Figure 4 : Ajout assisté d'un type utilisant l'interface VisualFigaro.

Sur la figure précédente on peut remarquer que pour déclarer un type il n'y a que deux syntaxes possibles. Dans le cas du type « nœud » la première proposition est suffisante car ce type n'hérite d'aucun autre type. Il suffit alors de cliquer sur le menu intitulé « TYPE nom ; ». Cette phrase est mise dans le presse-papier et il ne reste plus qu'à la coller dans la partie texte du logiciel et à remplacer le paramètre « nom » par « nœud ». Les types « sources », « recepteur » et « interrupteur » ont quant à eux la particularité de tous hériter du type « nœud » défini précédemment. Une exploration de la hiérarchie des prototypes proposés permet de s'apercevoir qu'il existe une définition prenant en compte l'existence d'un parent pour le type déclaré (Figure 5 gauche).

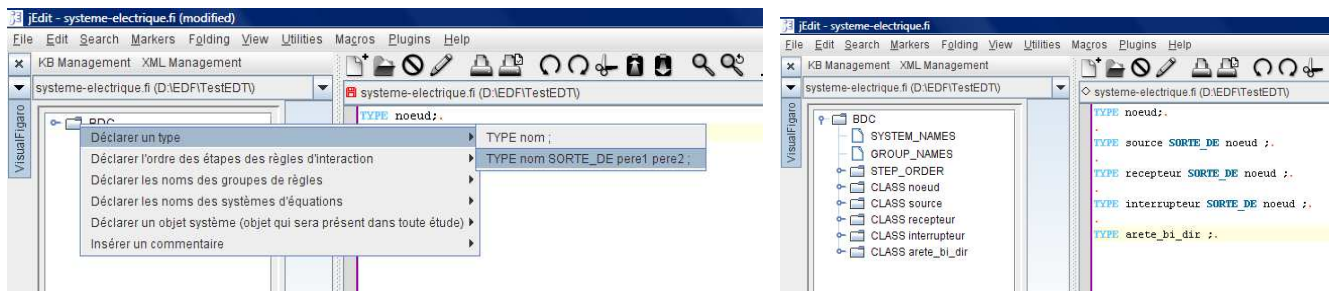


Figure 5 : Ajout des types héritant de « nœud » et obtention du premier squelette de base de connaissances avec affichage de l'arborescence syntaxique.

Il suffit donc de cliquer sur le menu pour placer la proposition dans le presse-papier, la coller trois fois dans la partie texte et remplacer les prototypes de noms par les noms réels des composants. Enfin la création du dernier type, « arete_bi_dir », s'effectue de la même manière que pour le type nœud. Maintenant que les principaux types se trouvent dans la base de données il faut mettre en place les interfaces qui vont permettre aux différents objets de communiquer entre eux. Cependant, afin de pouvoir utiliser la hiérarchie disponible dans le plugin et les prototypes proposés, il faut mettre à jour la hiérarchie en appuyant sur le bouton vertical marqué « <= ». L'exploration de l'arbre fera alors se découvrir l'architecture de la figure 5 (droite).

Les types déclarés précédemment sont maintenant bien présents et pris en compte. Toutes les suggestions de structures de code pour le "corps" des types sont à présent disponibles. L'ajout d'interface peut donc se faire de manière assistée dès à présent.

Ajout des interfaces

Les interfaces servent à représenter les connexions entre les composants et ainsi permettent de les faire interagir. Leur déclaration est la dernière étape indispensable concernant le code Figaro avant de pouvoir charger la base de connaissances dans KB3 et l'utiliser pour saisir des assemblages de composants (pour l'instant dénués de tout comportement), donc des structures de systèmes. Les interfaces correspondent aux points de contacts entre deux composants. Une interface appartient à un type particulier et a pour caractéristiques principales le type de composants que l'on peut y connecter et le nombre de composants de ce type qu'elle peut contenir. Toutes ces informations peuvent aisément être devinées par un débutant en consultant la documentation associée à « Interface » dans le menu associé. Une interface étant une partie d'un type, un clic droit sur le type en question, par exemple « CLASS nœud » permet de faire apparaître le menu déroulant visible sur la figure 6 (gauche).

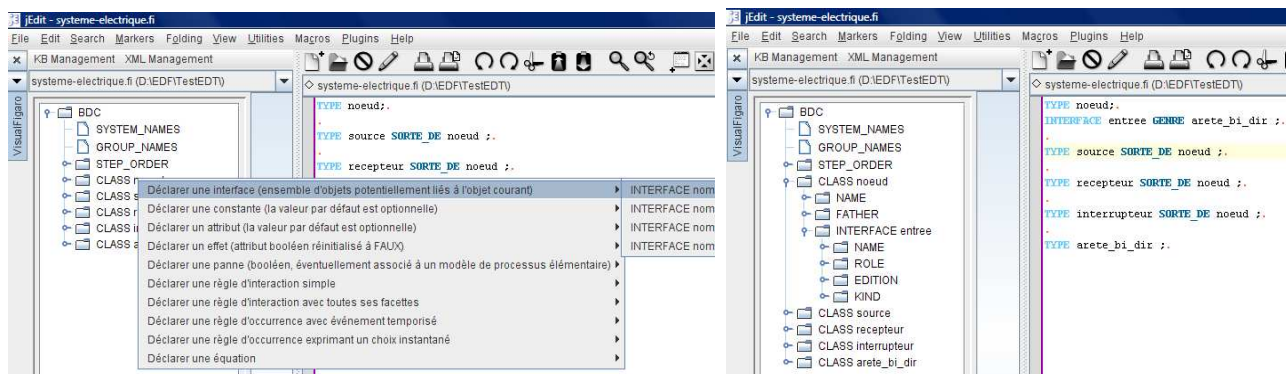


Figure 6 : Menu de complétion pour les interfaces d'un type et arborescence obtenue après l'ajout de l'interface « entree » au type "noeud".

L'interface à créer étant d'un type simple la première suggestion « INTERFACE nom GENRE nom_de_type » est suffisante. Encore une fois un clic sur le menu place ce prototype dans le presse-papier afin de permettre de le coller dans la partie texte juste après le type que l'on est en train de traiter. L'interface que l'on est en train de construire étant vouée à connecter un composant avec un conducteur de courant on remplacera le champ « nom » par le nom de l'interface par exemple « entree » puis le « nom_de_type » par la valeur « arete_bi_dir » car tout composant doit se brancher à un fil pour se connecter aux autres. Le code final obtenu après ajout de l'interface « entree » pour le noeud se trouve dans partie droite de la figure 6. Après l'ajout de toutes les interfaces nécessaires se clôt la partie édition de code FIGARO strictement nécessaire au lancement de KB3.

Création d'un premier fichier de paramétrage de l'IHM de KB3

Lors de l'édition d'une base de connaissances destinée à KB3 la partie graphique est paramétrée par un fichier XML dont l'extension est modifiée en « .bdc ». Avant la création de Visual Figaro l'édition de ce fichier était manuelle et donc fastidieuse étant donné la taille du fichier, même si elle s'appuyait sur un éditeur de XML performant. Ce n'est plus le cas car tout se passe désormais par le biais d'une interface graphique de type boîte de dialogue avec des menus déroulants, des cases à cocher, des champs à remplir. L'interface assure constamment la conformité du fichier XML au schéma qu'il doit respecter et ne propose dans les listes de choix que des options cohérentes avec le contenu du fichier FIGARO, ce qui peut faire gagner beaucoup de temps sur toutes les phases de construction : du prototypage à l'édition avancée d'une base.

L'accès à l'édition du fichier XML se fait par le biais du bouton « XML management » situé au dessus de l'arbre syntaxique. Cette action fait alors apparaître une fenêtre de choix (Figure 7 gauche). Celle-ci permet de choisir entre la création d'une interface graphique minimale de manière automatique ou bien la création entièrement manuelle de l'interface. Pour les débutants il est recommandé de commencer avec une interface déjà remplie. C'est le choix qui sera effectué par la suite. Quel que soit le choix effectué, la fenêtre qui s'affiche ensuite est celle représentée sur la figure 7 (droite).

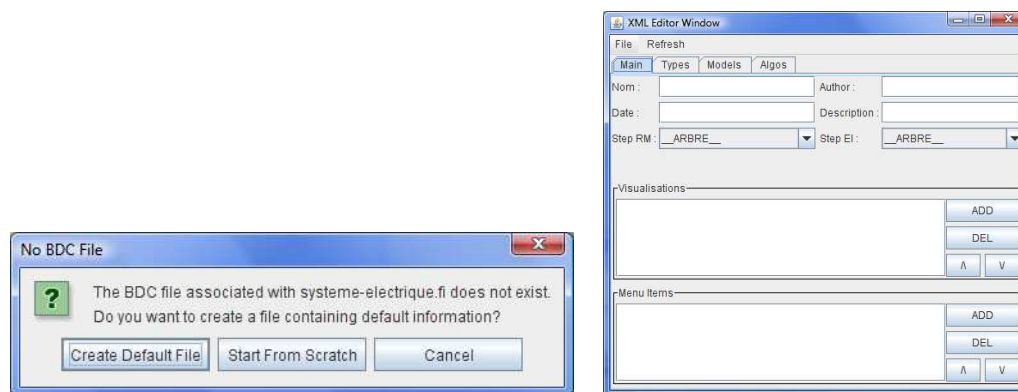


Figure 7 : Fenêtre de confirmation d'entrée dans l'interface d'édition assistée du fichier de configuration de l'interface graphique de KB3 représentée dans la partie droite de la figure.

C'est cette fenêtre qui sera appelée « fenêtre principale » dans le reste de l'article. Elle est le point d'accès à tous les principaux menus qui permettent de configurer l'aspect et les options disponibles dans KB3, par le biais des 4 onglets de la partie supérieure décrits ci-dessous.

- **Main** : Onglet donnant accès aux informations générales relatives à la base.
- **Types** : Onglet donnant accès aux informations relatives à chaque type en particulier.
- **Models** : Onglet donnant accès au paramétrage des menus de traitements autorisés pour la base de connaissances (génération d'arbre de défaillance, lancement de FIGSEQ ou YAMS etc.).
- **Algos** : Onglet donnant accès à la définition d'algorithmes dits « d'objets déduits » qui permettent de construire ou modifier automatiquement des objets FIGARO à partir de ceux saisis par l'utilisateur. **La plupart des bases de connaissances n'utilisent pas cette possibilité.** Mais dans certains cas, en créant de tels algorithmes (à l'aide du langage FIGARO auquel des opérateurs de création, modification, parcours d'objets ont été ajoutés) on augmente considérablement la puissance de modélisation du langage FIGARO de base. Cela permet en outre des optimisations très efficaces du modèle à traiter (par exemple, par agrégation automatique de défaillances ayant les mêmes conséquences sur le système).

Remarque : Etant donné la grande simplicité du contenu des onglets Main et Models (ce sont ceux que l'on remplit en premier), et le caractère facultatif de l'onglet Algos, dans la suite de l'article, seul le contenu de l'onglet Types sera évoqué.

Affectation de la catégorie graphique nœud ou lien aux types FIGARO

Chaque type peut être vu comme un lien ou un nœud dans un le graphe représentant le système. Si un type est considéré comme un lien dans le graphe il prendra la forme d'une flèche. Les interfaces de départ seront remplies par le composant situé au début de la flèche et celles d'arrivée par celui à la fin. Dans cet exemple le type « arete_bi_dir » qui représente un fil sera évidemment considéré dans l'interface graphique comme un lien. A l'inverse les types « nœud », « source » ou encore « recepteur » seront considérés comme des nœuds du graphe et représentés sous la forme d'une icône.

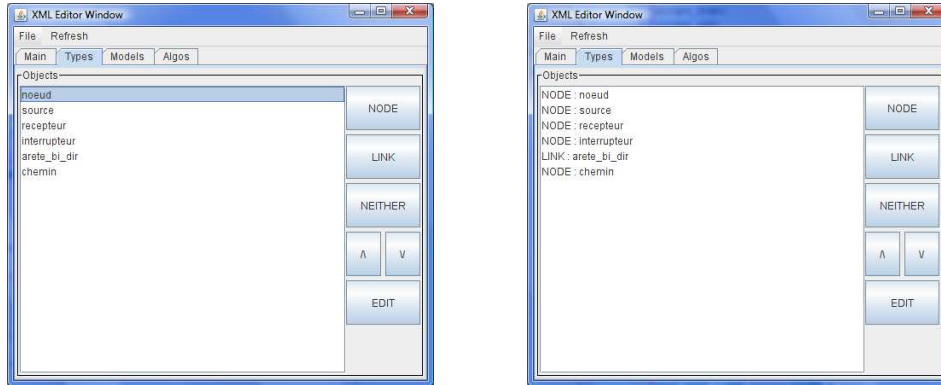


Figure 8 : Fenêtre principale sur l'onglet édition de type ; sur la partie droite les types avec leurs modes de représentation

L'interface permet de choisir entre une des deux représentations en sélectionnant le type en question et en cliquant sur le bouton « node » ou bien « link » selon le choix effectué. Cette action a pour effet d'ouvrir une des deux fenêtres que l'on trouve sur la figure 9.

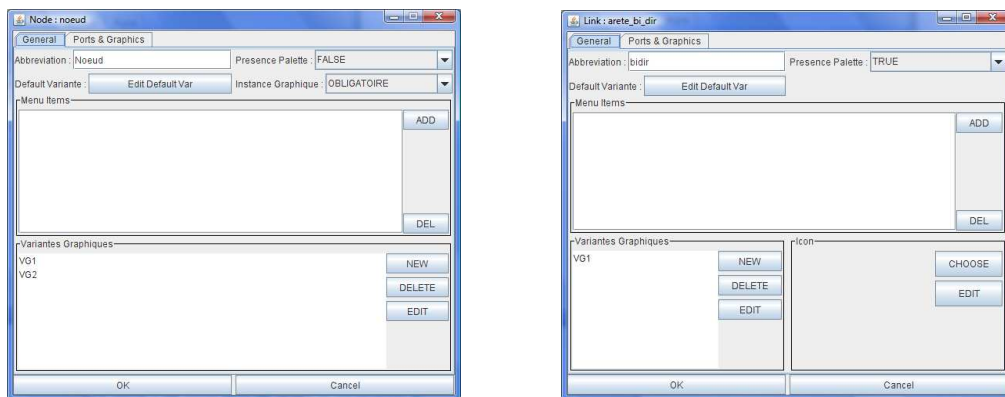


Figure 9 : Fenêtre d'édition de nœud (gauche) et de lien (droite)

Définition des contraintes sur les connexions possibles et des règles de remplissage d'interfaces

Pour établir un lien entre le comportement graphique de KB3 (une fois la base de connaissances chargée) et la création d'un modèle FIGARO il faut spécifier, via des « règles de remplissage d'interfaces » définies dans le fichier XML, à quelle action sur le modèle FIGARO correspond le fait de connecter un lien sur un nœud. En outre, afin de rendre l'interface « intelligente » il faut pouvoir lui spécifier, par exemple, que lorsqu'un utilisateur tente de brancher un fil électrique sur une pompe, ce fil doit aller « s'accrocher » sur le point de l'icône de la pompe qui correspond à l'arrivée de l'alimentation électrique, et non à l'arrivée d'eau !

Les interfaces définies en FIGARO pour chaque composant doivent donc être complétées par des informations supplémentaires pour pouvoir être utilisées au sein d'une interface graphique. Les informations attachées aux interfaces d'un type sont disponibles à partir de sa fenêtre dédiée en cliquant sur l'onglet « Ports & Graphics » (Cf. Figure 9).

Par exemple, dans la base de connaissances sur les systèmes électriques, comment doit se traduire le branchement d'une arete_bi_dir sur un nœud ? Le nœud propose deux interfaces entrée et sortie qui acceptent toutes des « arete_bi_dir ». L'arete_bi_dir propose deux interfaces départ et arrivée qui acceptent toutes des nœuds. Le comportement à décrire en s'appuyant sur les menus déroulants proposés par Visual Figaro est le suivant :

- Si le nœud se situe au début de l'arête alors remplir l'interface sortie du nœud par l'arête et l'interface départ de l'arête par le nœud.
- Si le nœud se situe à la fin de l'arête alors remplir l'interface entrée du nœud par l'arête et l'interface arrivée de l'arête par le nœud.

Chargement dans KB3 et saisie de petits systèmes de test

A ce stade, le chargement de la base de connaissances dans KB3 permet de s'assurer de l'absence de toute incohérence dans ce qui a été défini jusque là et de vérifier par la saisie de petits assemblages de composants (qui pourront servir par la suite de systèmes tests) que les règles de remplissage des interfaces fonctionnent comme on le souhaite. Pour l'instant, les icônes n'ayant pas été définies, elles sont toutes remplacées par des carrés dans l'IHM de KB3.

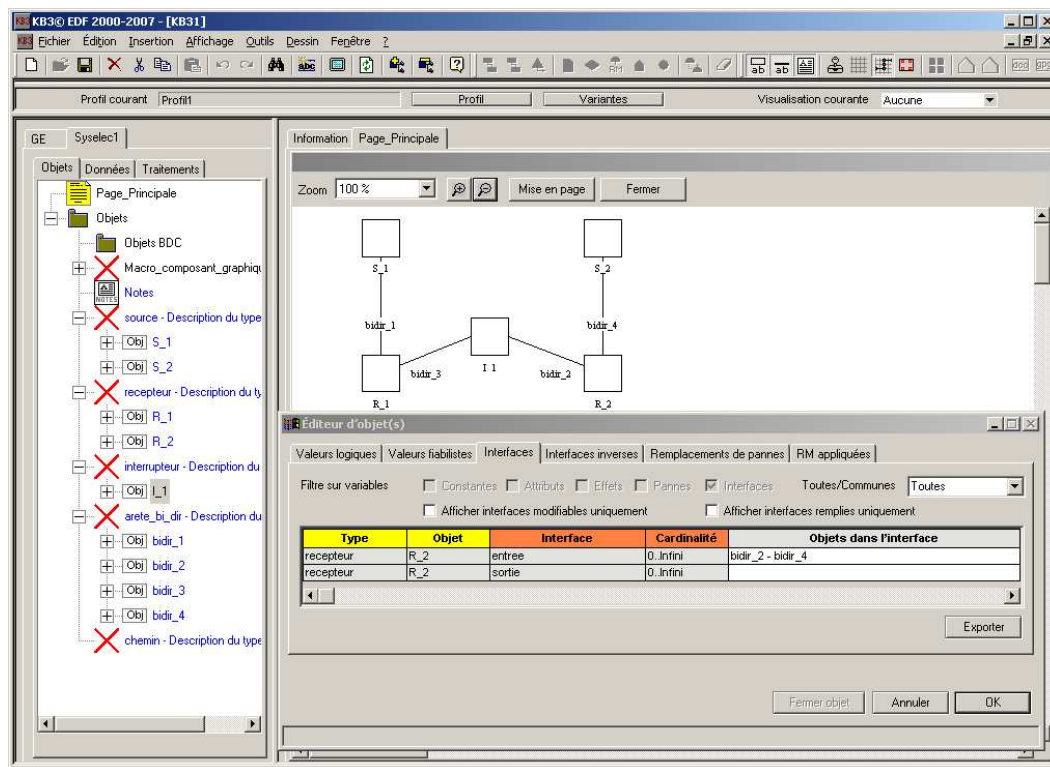


Figure 10 : Construction d'un système test dans KB3 grâce au squelette de la base de connaissances

Création des premières règles

Les composants sont donc maintenant des boîtes vides, que l'on peut accrocher les unes aux autres, à condition de respecter certaines contraintes. Pour les faire interagir il faut écrire des règles en langage FIGARO, sans oublier de déclarer les constantes et variables qu'elles manipulent. Elles permettent de propager les conséquences des modifications qui ont été effectuées sur un composant à tous les autres composants entraînant ainsi l'évolution du système vers un nouvel état. Ces règles ayant beaucoup de répercussions sur le système il est fortement recommandé de tester fréquemment la base de connaissances avec KB3 afin de s'assurer par des simulations interactives que le comportement est conforme à ce que l'on attend.

La rédaction de ces règles est la partie la plus complexe de la création de la base de connaissances. Le lecteur désireux de connaître les détails de la présente base devra se référer à l'article [2] qui donne l'intégralité du code FIGARO. Cependant les prototypes proposés par Visual Figaro permettent de retrouver facilement la syntaxe des constructions un peu complexes telles que les expressions avec des quantificateurs. Il en va de même pour la déclaration des variables (qui se subdivisent en effets, pannes et attributs), et des constantes que l'on trouve dans les règles. La figure 11 permet d'illustrer une déclaration d'effet par le biais de l'interface graphique.

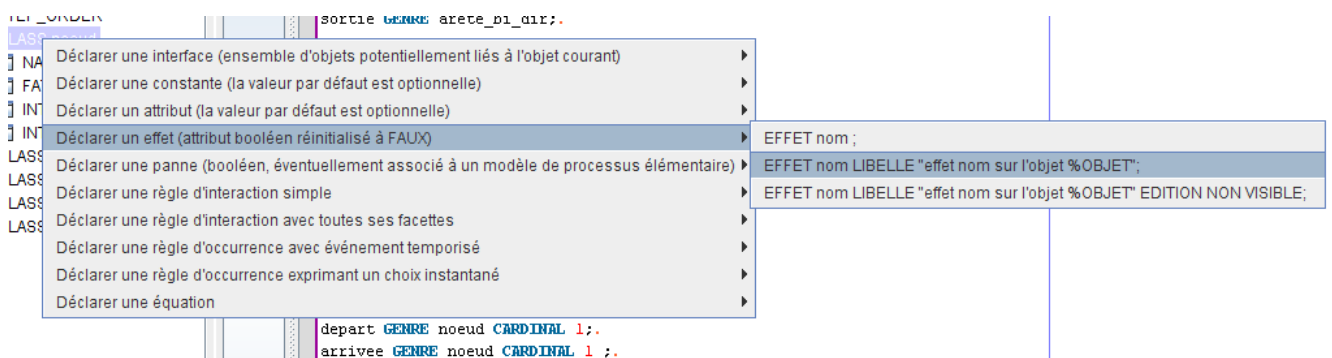


Figure 11 : Ajout d'un effet au type "Nœud" et modification de la base de connaissances en conséquence.

Boucles de test dans KB3 et révision des règles

Les règles initiales ayant été écrites, l'essentiel du développement de la base va consister à alterner les deux activités suivantes :

- Test au sein de KB3 par simulation interactive (ou éventuellement en lançant d'autres traitements)
- Création ou modification de règles

La prochaine étape a pour but de rendre l'utilisation de la base de connaissances plus conviviale, de façon à créer un véritable outil dédié à l'étude d'une catégorie de systèmes.

Améliorations de l'interface graphique

L'interface graphique visible dans KB3 est une partie à ne pas négliger car c'est dans cet outil que les utilisateurs de la base de connaissances passeront le plus de temps. Les items doivent être clairs et parlants pour ces utilisateurs, surtout s'ils sont différents de ceux qui ont écrit la base de connaissances. La partie la plus visible dans ce logiciel est constituée par les icônes qui représentent les types et leurs instantiations. Leur paramétrage s'effectue au sein de l'interface graphique dont la fenêtre principale est représentée figure 7 (droite), en ouvrant dans un premier temps l'onglet Types.

Ajout de variantes graphiques

Les variantes graphiques sont l'ensemble des caractéristiques qui permettent de définir l'aspect graphique d'un type une fois qu'il est placé dans la fenêtre d'édition. Les caractéristiques des nœuds et des liens étant très différentes, le contenu de leurs variantes graphiques l'est aussi. Par exemple on trouve respectivement :

- Nœud : icône à afficher, description, point d'attache des liens sur l'icône.
- Lien : épaisseur et type du trait, couleur.

Toutes ces informations sont disponibles dans les fenêtres d'édition des nœuds et des liens. Par exemple, dans la base de connaissances actuelle, les sources doivent être reconnaissables par l'icône de source présente dans le dossier icônes de la base de connaissances. Il faut donc ouvrir la fenêtre d'édition du type en question en utilisant le bouton nœud. Une fois cette opération effectuée l'édition de la variante graphique par défaut se fait en poussant le bouton « Edit Default Var ». Une fois cette fenêtre remplie le type « source » sera représenté par l'icône « S » dans tous schémas dessinés dans KB3 avec cette base de connaissances.

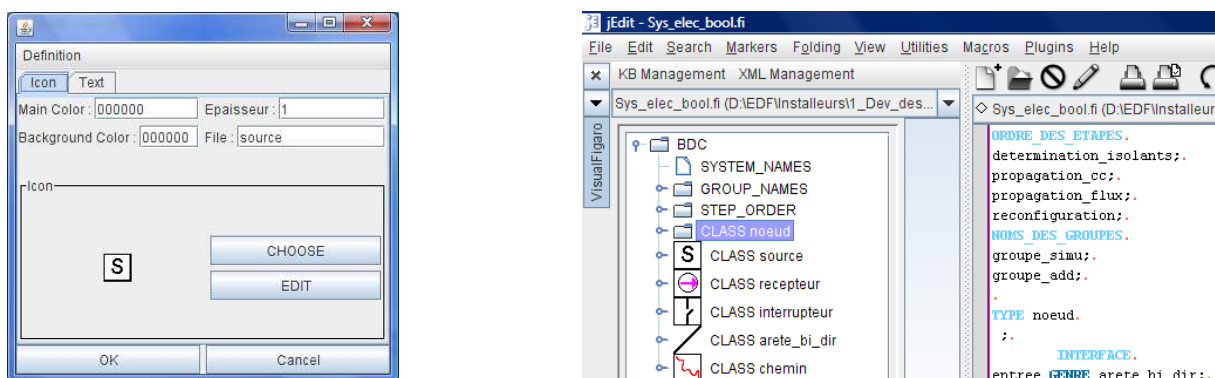


Figure 12 : Edition d'une variante graphique et mise à jour de l'arborescence avec les apparitions des icônes

L'utilisation de tous les champs ne sera pas détaillée ici mais elle peut être trouvée dans la documentation de KB3. La variante graphique modifiée va impliquer la mise à jour de l'arborescence de VisualFigaro. L'icône de « dossier » d'origine est remplacée par celle de la variante graphique. Cela confirme le fait qu'à partir de ce moment dans KB3 les sources seront représentées par cette icône (figure 12 droite).

Validation finale avec KB3

La validation de la base de connaissances s'effectue tout en la construisant à l'aide des multiples chargements dans KB3 par exemple lors de la modification des règles. Cependant la validation finale est plus poussée que toutes les précédentes car elle va jusqu'à l'utilisation des outils qui traitent le modèle de système obtenu. Il faut prendre soin d'analyser les résultats de ces outils afin de remarquer toute anomalie qui ne serait pas en accord avec les dires des experts.

CONCLUSION

Dans cet article, nous avons montré comment, grâce à une méthode simple et progressive, il est possible en partant d'une "feuille blanche" de construire très rapidement un outil spécialisé pour l'étude d'une certaine catégorie de systèmes, en s'appuyant sur les progiciels de la plate-forme outils KB3.

Le dernier né de ces outils, Visual Figaro, est celui qui permet d'accompagner la démarche que nous avons écrite en déchargeant à tout moment le développeur de bases de connaissances des aspects secondaires, mais susceptibles de générer d'importantes pertes de temps : connaissance de la syntaxe FIGARO, de la structure du fichier de paramétrage de l'interface graphique générique de KB3, cohérence entre ces deux fichiers, navigation dans l'ensemble de la base de connaissances...

Références

- [1] M. Bouissou, H. Bouhadana, M. Bannelier, N. Villatte. Knowledge modelling and reliability processing : presentation of the FIGARO language and associated tools, Safecomp'91, Trondheim (Norvège), novembre 1991.
- [2] M. Bouissou, C. Seguin. Comparaison des langages de modélisation AltaRica et FIGARO, Im15, Lille, octobre 2006.
- [3] M. Bouissou, S. Humbert, S. Muffat, N. Villatte
KB3 Tool: Feedback On Knowledge Bases, ESREL 2002, Lyon (France), mars 2002.
- [4] M. Bouissou, J.C. Houdebine
Inconsistency Detection In KB3 Models, ESREL 2002, Lyon (France), mars 2002.
- [5] <http://rdsoft.edf.fr> - lien vers Plateforme KB3.